

DOCUMENTAÇÃO TÉCNICA

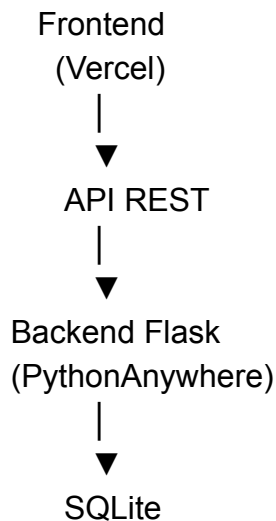
Plataforma ELEVA

Arquitetura Geral da Solução

O ELEVA foi desenvolvido com uma arquitetura desacoplada (*Decoupled Architecture*), abordagem amplamente utilizada em aplicações web modernas que separa claramente as responsabilidades entre interface, regras de negócio e persistência de dados.

Nesse modelo, frontend e backend operam como aplicações independentes, comunicando-se exclusivamente por meio de uma API REST. Essa separação permite maior flexibilidade de implantação, manutenção simplificada e evolução independente de cada camada.

Arquitetura simplificada



A arquitetura foi estruturada em três componentes principais:

- **Camada de apresentação (Frontend)**
- **Camada de processamento e regras de negócio (Backend)**
- **Camada de persistência de dados**

Infraestrutura e Hospedagem

Frontend: Vercel

O frontend foi hospedado no Vercel, plataforma de hospedagem e distribuição de aplicações web com foco em performance. A escolha foi motivada pela CDN global da plataforma: os arquivos são entregues pelo servidor geograficamente mais próximo do usuário, reduzindo a latência e garantindo carregamento rápido mesmo em conexões limitadas.

No ELEVA, o frontend é responsável pela renderização da interface, navegação entre páginas, validações no cliente e comunicação com a API.

Backend: PythonAnywhere

O backend foi implementado em Python utilizando o microframework Flask e hospedado no PythonAnywhere. A escolha do Flask foi deliberada: trata-se de um framework minimalista que permite criar APIs REST sem a complexidade estrutural de frameworks mais robustos, como o Django. Para o escopo arquitetural do ELEVA, a tecnologia ofereceu equilíbrio entre simplicidade, flexibilidade e velocidade de desenvolvimento.

O PythonAnywhere foi escolhido por sua facilidade de implantação para aplicações Python, executando continuamente o servidor WSGI necessário para disponibilização das APIs.

O backend centraliza toda a lógica operacional da plataforma, incluindo processamento de regras de negócio, autenticação de usuários, comunicação com o banco de dados e controle dos mecanismos de segurança e rastreabilidade.

Comunicação entre Serviços

Como frontend e backend estão hospedados em domínios distintos, a comunicação ocorre via requisições HTTP utilizando o padrão REST, com respostas estruturadas em formato JSON.

Para permitir essa comunicação de forma segura, foi implementada a configuração CORS no backend, autorizando explicitamente requisições provenientes do domínio do frontend hospedado no Vercel.

Linguagens e Tecnologias

HTML5

O HTML5 define a estrutura semântica de todas as páginas da aplicação.

CSS3

O CSS3 é responsável pela estilização, responsividade e identidade visual da plataforma. O projeto utiliza Flexbox, Grid e variáveis nativas () para padronização visual, permitindo alterar toda a paleta de cores da aplicação a partir de um único ponto central do código.

JavaScript Vanilla

A utilização de JavaScript puro foi uma decisão técnica deliberada. Em vez de utilizar frameworks como React ou Vue, optou-se pelo JavaScript Vanilla para reduzir dependências externas, simplificar a arquitetura do frontend e manter baixo custo computacional durante a execução da aplicação.

O JavaScript é responsável por capturar interações do usuário, realizar chamadas à API por meio da função fetch() e atualizar dinamicamente a interface sem necessidade de recarregamento da página.

Python com Flask

O Python, utilizando o framework Flask, processa todas as regras críticas de negócio no servidor, fora do alcance do cliente. É o backend que gera tokens de confirmação de serviço, calcula e retém valores em escrow e controla o repasse para as instituições parceiras.

Nenhuma dessas operações pode ser manipulada diretamente pelo navegador do usuário.

SQLite

O SQLite armazena todos os dados da plataforma em um único arquivo no servidor. A escolha foi intencional: para o volume de operações previsto no estágio atual do ELEVA, a utilização de um sistema gerenciador de banco de dados mais robusto, como PostgreSQL, representaria aumento de complexidade operacional incompatível com o estágio atual do projeto.

O SQLite oferece simplicidade operacional, facilidade de depuração e desempenho adequado para a lógica de escrow e rastreabilidade implementada na plataforma.

Fluxo do Escrow

O pagamento nunca é transferido diretamente ao profissional. O fluxo opera em quatro etapas:

1. O cliente confirma o pagamento. O JavaScript converte o valor do formato brasileiro (1.500,00) para o formato numérico utilizado internamente antes da realização dos cálculos.
2. O backend gera um token numérico de quatro dígitos e atualiza o status da solicitação para **Pago (Retido)**.
3. Para iniciar o serviço, o profissional insere presencialmente o token de validação, confirmando sua presença no local.
4. Após a conclusão e avaliação do cliente, o status da transação é alterado para **Concluído**, liberando o valor correspondente.

Autenticação e Segurança

As senhas são armazenadas utilizando hash criptográfico por meio da biblioteca `werkzeug.security`, nunca em texto simples.

O sistema de autenticação permite acesso tanto por e-mail quanto pelo ID de ativação do profissional.

Todas as operações relacionadas ao mecanismo de escrow e rastreabilidade são processadas exclusivamente no backend, impedindo manipulação direta pelo cliente.

Decisões Técnicas Relevantes

Tratamento de Busca

O sistema de busca captura o termo digitado, remove espaços excedentes utilizando `trim()` e injeta o valor na URL por meio de *query strings* utilizando `encodeURIComponent()`, evitando que caracteres especiais comprometam a requisição.

Processamento Financeiro

O cálculo financeiro converte valores monetários do padrão brasileiro (1.500,00) para o formato numérico utilizado internamente, garantindo precisão no cálculo das taxas institucionais antes do envio dos dados ao backend.

Painéis Administrativos

Os painéis de cliente, profissional e instituição utilizam uma abordagem SPA simplificada, na qual as seções são exibidas ou ocultas dinamicamente via CSS e JavaScript, reduzindo recarregamentos e melhorando a experiência do usuário sem introduzir complexidade arquitetural adicional.

Controle de Acesso

O acesso aos painéis verifica a existência de credenciais armazenadas no localStorage antes da renderização do conteúdo. Caso o usuário não esteja autenticado, ocorre redirecionamento automático para a tela de login.

Interface de Negociação

O sistema de chat foi projetado utilizando padrões de interação inspirados em aplicativos de mensagens instantâneas, reduzindo a curva de aprendizado e aumentando a familiaridade da interface para os usuários.